Personalized Content Discovery

Technical Report: Recommendation System Challenge

Executive Summary

This report details the development of a personalized recommendation system using the Netflix Prize Dataset, designed to learn user preferences and generate relevant content recommendations. We evaluated Memory-based (User/Item KNN), Matrix Factorization (SVD), and Deep Learning (Neural CF) models. **SVD provided the best balance** of predictive accuracy, robust coverage, and computational efficiency, achieving strong performance on both RMSE and the MAP@10 competition metric.

1. Introduction & Motivation

Recommendation systems influence user engagement and retention across platforms by understanding preferences and delivering relevant content. The goal of this project is to develop a highly personalized recommendation engine capable of predicting user ratings and generating Top-K recommendations. This report covers the end-to-end pipeline: from Exploratory Data Analysis (EDA) and data processing, to model development, evaluation, and interactive deployment.

---

2. Exploratory Data Analysis & Business Insights

Our analysis of the dataset revealed several critical insights that heavily informed our modeling strategy:

- **Extreme Sparsity (98%+ Empty):** The vast majority of users have interacted with a tiny fraction of the 17.7K movies. This sparsity makes memory-based collaborative filtering (KNN) highly susceptible to neighborhood unreliability.
- **Skewed Rating Distribution:** Ratings are heavily skewed toward 3, 4, and 5 stars. Users generally rate content they already suspect they will like.
- **Power Law in Popularity:** A small percentage of movies generate the majority of ratings. This "long-tail" distribution means models must explicitly combat popularity bias to surface relevant niche content.
- **Temporal Growth:** Interaction volume exploded in 2004–2005. To simulate a real-world scenario, we employed a temporal train-test split rather than a random split.

---

3. Data Processing & Methodology

Data Pipeline

Raw data chunks were iteratively parsed to extract `user_id`, `movie_id`, `rating`, and `date`. To maintain computational feasibility, we sample the user base (e.g., 5% to 10% fractions) while maintaining the original temporal integrity of interactions.

Train-Test Splitting

We implemented a **Temporal Split (80/20)**. Ratings are sorted chronologically per user, and the first 80% of their historical interactions are placed in the training set, mimicking the real-world task of predicting future interactions based on past behavior.

Target Definition

For the MAP@10 evaluation metric, we defined a "relevant" recommendation as any item the user rated ≥ 3.5 stars in the test set.

---

4. Model Architectures

We developed and compared three distinct recommendation paradigms:

A. Matrix Factorization (SVD)

We implemented SVD to discover latent features explaining user-item interactions. SVD handles sparsity exceptionally well by mapping both users and items to a shared lower-dimensional latent space (factors=100, epochs=20).
Complexity: $O(N_{epochs} \times |R| \times f)$ where $|R|$ is the number of ratings and f is the latent dimension.

B. Memory-Based Collaborative Filtering (KNN)

We implemented both User-based and Item-based KNN using Cosine and Pearson similarities. While highly interpretable, KNN struggles computationally at scale.
Complexity: $O(|U|^2)$ or $O(|I|^2)$ to compute the similarity matrix, making it memory-bound for large datasets.

C. Neural Collaborative Filtering (NeuMF)

We built a dual-path deep learning model combining Generalized Matrix Factorization (GMF) and a Multi-Layer Perceptron (MLP) built dynamically in PyTorch. This captures complex, non-linear relationships.
Complexity: Highly parallelizable via GPU, but requires extensive hyperparameter tuning (Learning Rate, Dropout) to prevent overfitting on sparse users.

---

5. Evaluation & Comparison

We evaluated the models on predictive accuracy (RMSE, MAE) and ranking quality (MAP@10).

Model	RMSE	MAE	MAP@10	Train Time	Strengths / Weaknesses
SVD	**0.895**	**0.710**	**0.142**	**Fast**	**Robust baseline, handles sparsity well. Lacks non-linear capability.**
KNN (User)	1.012	0.825	0.085	Slow	Highly interpretable, but extremely slow inference and poor coverage.
Neural CF	0.932	0.741	0.128	Med (GPU)	Captures complex patterns, but prone to overfitting without strict tuning.

MAP@10 Methodology

1. We sort all unseen items by their predicted rating for a specific user.
2. We take the Top 10 highest-predicted items.
3. If an item appears in the user's ground-truth test set with a rating ≥ 3.5, it is a "hit".
4. We compute Average Precision for the user based on hit positions. MAP is the mean of these APs across all users with at least one relevant item in the test set.

---

6. Recommendation Quality & Analysis

Success Cases

For "Mainstream Users" with well-defined genre preferences (e.g., Sci-Fi/Action), the SVD model successfully curates highly relevant Top-10 lists. For example, a user who highly rated *The Matrix* and *Terminator 2* was accurately recommended *Aliens* and *Blade Runner*, resulting in a perfect AP@10 score.

Failure Cases

- **Cold Start Users:** Users with fewer than 5 ratings receive highly generic recommendations (mostly global blockbusters). SVD defaults to item biases, while KNN fails completely.
- **Popularity Bias:** For casual users, the system occasionally over-recommends extremely popular titles (like *The Shawshank Redemption* or *Lord of the Rings*) regardless of their niche preferences.

Explainability

Recommendations are explained to the user using the underlying model logic. For KNN, we surface the fact that "Users with similar tastes enjoyed this." For SVD, we rely on latent trait similarities between the recommended item and their highest-rated history.

---

7. Production System & Deployment

The models are wrapped in an interactive `Streamlit` dashboard (`dashboard/app.py`). The dashboard allows stakeholders to:

1. Select an arbitrary user and generate Top-10 predictions live.

2. View the user's historical ratings to validate the recommendations.
3. Explore the distribution and similarity clusters of specific movies.
4. Compare model evaluation metrics directly in the UI.

To ensure computational viability, the prediction pipeline is fully vectorized using `torch.no_grad()` and batch processing, generating recommendations in under 500ms.

---

8. Conclusion & Future Work

The SVD matrix factorization approach emerged as the superior model for this dataset, easily outperforming KNN in both speed and accuracy, while providing a stronger baseline than the un-tuned Neural CF model.

Future Work

1. **Hybridization:** Combine the collaborative filtering signal from SVD with a content-based model utilizing movie genres and cast (to solve the cold-start problem).
2. **Diversity Penalty:** Implement a post-processing re-ranker to penalize overly popular blockbusters and increase novelty.
3. **Advanced NeuMF Tuning:** Further tune the Neural CF model with implicit feedback (BPR Loss) to optimize directly for ranking rather than RMSE.
